Action Risk Tiering: a Scoring Model, Worked Out by Hand

Turning a fuzzy "is this action risky?" into one reproducible number — and a tier you can audit — for three refund-agent actions, counted by hand.

What this document does. It takes the four-tier policy from Module 4.7 — AUTO / NOTIFY / APPROVE / REFUSE — and replaces the hand-wavy "use your judgement" with an explicit scoring model. Each action is scored on the module's own three criteria (*reversibility*, *blast radius*, *regulatory exposure*), each axis normalized to $[0, 1]$, weighted, and summed into a single risk score in $[0, 1]$; thresholds map the score to a tier. We work three concrete refund-agent actions — `send email`, `issue $40 refund`, `delete account` — with explicit raw values and show one action crossing a threshold. Every number is reproduced by the Python program in Appendix A, so the scoring is exact and self-consistent.

Where this sits. This is **Topic 2 of Module 4.7** (Human-in-the-Loop & Governance) — the function `classify_tier()` the approval node calls before it decides whether to pause. Its companions: Topic 1 (the escalation ladder that catches everything this model routes to APPROVE) and Module 4.6 (the role scopes that enforce who may release each tier).

Concepts covered, in order: three risk axes · per-axis normalization to $[0, 1]$ · a weighted-sum score $s = \sum w_k x_k$ · threshold map to four tiers · three worked actions · a threshold-crossing (the \$600 refund) · sensitivity: the weight dial and the threshold dial · why a reproducible number is auditable policy.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: three axes, one score, four tiers

The module says every tool the agent can call "belongs in exactly one tier," and lists four classification criteria. We make that operational with three numeric axes (folding the module's "user-trust / surprise" criterion into regulatory-and-expectation exposure for a clean three-axis model). Each axis is measured in its own natural unit, then normalized onto $[0, 1]$ by dividing by a cap:

Axis	Raw unit	Normalizer (cap)
Reversibility	hours to undo	$x_{\text{rev}} = \min(\text{hours}/24,\ 1)$
Blast radius	dollars (or users) at risk	$x_{\text{blast}} = \min(\$/1000,\ 1)$
Regulatory exposure	level 0–3 ordinal	$x_{\text{reg}} = \min(\text{level}/3,\ 1)$

The weights and thresholds. The three normalized axes are combined with weights that sum to 1, and the resulting score is cut into tiers:

Weights	$w_{\text{rev}} = 0.30, \quad w_{\text{blast}} = 0.40, \quad w_{\text{reg}} = 0.30$
Tier thresholds	AUTO $< 0.10 \le$ NOTIFY $< 0.35 \le$ APPROVE $< 0.70 \le$ REFUSE

Notation. $x_k \in [0,1]$ is the normalized value of axis k; w_k its weight; the risk score is $s = \sum_k w_k x_k \in [0,1]$. A higher score means a riskier action and a stricter tier.

Intuition

The whole model is one line: *score = weighted average of three normalized worries.* Normalization makes "$800" and "2 hours to undo" comparable — both become a number between 0 and 1, where 1 means "as bad as this axis gets." The weights say how much each worry counts; the thresholds say where worry becomes a gate. Nothing here is clever. The point is not sophistication — it is that the same action always gets the same number, and a reviewer can read the policy off a table instead of guessing.

2 Step 1 — normalize one action by hand

Take `issue $40 refund`. Raw values: it takes about 3 hours to claw back a refund (reverse the payment, notify the customer), it puts $40 at risk, and it touches payments, which we rate regulatory level 1 on the 0–3 scale. Normalize each axis:

$$x_{\text{rev}} = \min\left(\tfrac{3}{24}, 1\right) = 0.125, \qquad x_{\text{blast}} = \min\left(\tfrac{40}{1000}, 1\right) = 0.040, \qquad x_{\text{reg}} = \min\left(\tfrac{1}{3}, 1\right) = 0.333.$$

Now the weighted sum:

$$s = 0.30(0.125) + 0.40(0.040) + 0.30(0.333) = 0.0375 + 0.0160 + 0.1000 = 0.1535.$$

Since $0.10 \le 0.1535 < 0.35$, the $40 refund lands in NOTIFY. ✓ A small refund proceeds and is logged — the human sees it happened, but is not asked to gate it. The regulatory axis, not the dollar amount, is what lifts it out of AUTO.

Mini-example — this operation in isolation

Watch which axis dominates. For the $40 refund the three contributions are 0.0375, 0.0160, 0.1000 — the *regulatory* term (0.10) is two-thirds of the whole score, even though $40 is trivial money. Strip the payments flag (set reg = 0) and the score collapses to $0.0535 < 0.10$: pure AUTO. One ordinal flag moved the action a whole tier. This is why "does it touch payments / PHI / personal data?" is asked first — it is frequently the load-bearing axis, and it is the one a human can answer without measurement.

3 Step 2 — score all three actions

The same recipe applied to three actions spanning the range. Raw values and the resulting tier:

Action	hrs	$blast	reg	x_{rev}	x_{blast}	x_{reg}	$s \to$ tier
`send templated email`	0.5	0	0	0.021	0.000	0.000	0.0062 → AUTO
`issue $40 refund`	3.0	40	1	0.125	0.040	0.333	0.1535 → NOTIFY
`delete account`	24.0	800	3	1.000	0.800	1.000	**0.9200** → REFUSE

Sample check, `delete account`: it is effectively irreversible ($\geq$ 24h $\Rightarrow x_{\text{rev}} = 1$), affects \$800 of value / many user records ($800/1000 = 0.8$), and is maximally regulated ($3/3 = 1$). So $s = 0.30(1) + 0.40(0.8) + 0.30(1) = 0.30 + 0.32 + 0.30 = 0.92$. ✓ Above the 0.70 refuse line — the agent never does this; it is a human-only domain, exactly as the module's Tier 04 prescribes. The three actions fall cleanly into three different tiers, which is the model doing its job: separating a harmless read from a logged side-effect from a forbidden write.

4 Step 3 — an action crossing a threshold

The interesting cases live *on* a boundary. Hold the refund's reversibility (3h) and regulatory level (1) fixed, and raise only the dollar amount. The blast axis climbs, dragging the score across the NOTIFY → APPROVE line at 0.35:

Refund	$x_{\text{blast}} = \min(\$/1000, 1)$	**score** s	**tier**
\$40	0.040	0.1535	NOTIFY
\$300	0.300	0.2575	NOTIFY
\$600	0.600	**0.3775**	APPROVE

At \$600 the blast contribution is $0.40 \times 0.600 = 0.24$; added to the fixed 0.0375 (reversibility) and 0.10 (regulatory) it gives 0.3775, which clears 0.35 and tips the action into APPROVE. ✓ The implied policy: "refunds under \$ $\sim$ 550 self-serve; above that, a human signs off" — and that dollar break-even is not a magic constant typed into an `if`, it falls out of the score and the threshold. This is the boundary that produced the module's `request_4135 · refund $1,420 · APPROVE · L2`: well past \$600, deep in APPROVE and already escalated.

> **Watch out**
>
> A weighted sum is *compensatory*: a high score on one axis can be masked by low scores on the others. A \$5,000 refund that is instantly reversible and unregulated scores $0.40 \times 1.0 + 0 + 0 = 0.40$ — only just APPROVE, never REFUSE, no matter how large the dollar figure. If some axis must be able to *veto* regardless of the others (e.g. "anything touching PHI is at least APPROVE"), encode that as a hard floor *outside* the weighted sum — a `max()` with a per-axis minimum tier. The score is the default; the floor is the non-negotiable.

5 Step 4 — the governance dials: weight and threshold

The model has exactly two kinds of knob, and re-tiering an action means turning one of them. Both are policy — versioned, reviewable, auditable — not code buried in a function.

Dial 1 — the weight. Consider a refund-to-many action that is purely blast-heavy: 1h to undo ($x_{\text{rev}} = 0.042$), \$500 at risk ($x_{\text{blast}} = 0.5$), unregulated ($x_{\text{reg}} = 0$). Under the default weights it is NOTIFY; raise the blast weight to 0.70 (taking the others to 0.15) and the same action becomes APPROVE:

$$\text{default } (0.30/0.40/0.30): \quad s = 0.30(0.042) + 0.40(0.5) + 0.30(0) = 0.2125 \;\rightarrow\; \text{NOTIFY},$$

$$\text{blast-weighted } (0.15/0.70/0.15): \quad s = 0.15(0.042) + 0.70(0.5) + 0.15(0) = 0.3562 \rightarrow \text{APPROVE}. \checkmark$$

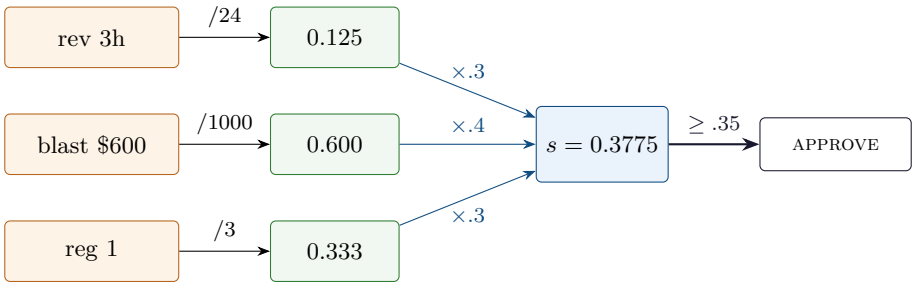
Dial 2 — the threshold. Keep the weights, move the line. The \$300 refund scores 0.2575. With the APPROVE cut at its default 0.35 it is NOTIFY; lower that cut to 0.25 — "we want a human on anything over \$250-ish" — and the *same* score now reads APPROVE:

$$s = 0.2575: \quad \text{cut } 0.35 \Rightarrow \textsc{notify}, \qquad \text{cut } 0.25 \Rightarrow \textsc{approve}. \quad \checkmark$$

Intuition

These two dials are the entire governance surface. Tightening after an incident is a one-line diff — bump a weight or drop a threshold — and because the score is reproducible, you can *replay* every past action through the new policy and see exactly which ones would have re-tiered, before you ship. "We made refunds stricter" stops being a Slack message and becomes a diff with a backtest. That auditability is the whole reason to turn a judgement call into a number.

6 The scoring pipeline, drawn



Brown raw values are normalized (green) by their caps, weighted and summed into the blue score, then thresholded into a tier. The picture *is* the \$600-refund row of the crossing table: $0.3775 \geq 0.35 \Rightarrow$ APPROVE.

7 Tiering cheat-sheet

Normalize an axis	$x_k = \min(\text{raw}_k/\text{cap}_k,\ 1)$
Risk score	$s = \sum_k w_k\, x_k, \qquad \sum_k w_k = 1$
Default weights	w_{rev}=0.30, w_{blast}=0.40, w_{reg}=0.30
Caps	24 h, \$1000, reg level 3
Tier map	AUTO$<$0.10$\leq$NOTIFY$<$0.35$\leq$APPROVE$<$0.70$\leq$REFUSE
Weight dial	change w_k (keep $\sum w_k = 1$) $\Rightarrow$ re-score
Threshold dial	move a cut $\Rightarrow$ re-tier at fixed score
Hard floor (optional)	per-axis minimum tier via `max`, outside the sum

8 An honest word about these numbers

The weights (0.3/0.4/0.3), the caps (24h, $1000, level 3), and the thresholds (0.10/0.35/0.70) are illustrative round choices, picked so the weighted sums are followable by hand. Your real weights encode *your* risk appetite; your real caps depend on what "maximum blast" means for your product (a B2B tool's $1000 is a consumer app's $50); your real regulatory scale may have ten levels, not four. Those numbers are policy, and policy is supposed to move. What is *not* illustrative is the structure: normalize each axis to a common $[0, 1]$ scale, take a weighted sum, and threshold — so that the same action always yields the same score and the same tier, and every change to weights or thresholds is a diff you can audit and backtest. A linear weighted sum is also deliberately the *simplest* defensible model; if your axes interact (some combination is worse than the sum of its parts) you will reach for a richer function, but you should start here, because a model a regulator can read in one line is worth more than a clever one nobody can.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the three worked actions, the threshold-crossing table, and both governance dials; change the weights, caps, or thresholds to encode your own policy and re-score every action against it.

```python
# tier_score.py -- reproduces every number in
# "Action Risk Tiering: a Scoring Model, by Hand".

# --- axis weights (the policy dial) ---
W_REV, W_BLAST, W_REG = 0.30, 0.40, 0.30 # sum to 1.0

# --- normalizers: map each raw axis onto [0,1] ---
REV_CAP = 24.0 # hours to undo; >= 24h counts as fully irreversible
BLAST_CAP = 1000.0 # dollars (or users) at risk; >= 1000 saturates
REG_CAP = 3.0 # regulatory level on a 0..3 ordinal scale

def n_rev(hours): return min(hours / REV_CAP, 1.0)
def n_blast(dollars): return min(dollars / BLAST_CAP, 1.0)
def n_reg(level): return min(level / REG_CAP, 1.0)

def risk_score(hours, dollars, reg, wr=W_REV, wb=W_BLAST, wg=W_REG):
    r, b, g = n_rev(hours), n_blast(dollars), n_reg(reg)
    return r, b, g, wr*r + wb*b + wg*g

# --- thresholds: score -> tier ---
T_NOTIFY, T_APPROVE, T_REFUSE = 0.10, 0.35, 0.70
def to_tier(s, t_notify=T_NOTIFY, t_approve=T_APPROVE, t_refuse=T_REFUSE):
    if s < t_notify: return "AUTO"
    if s < t_approve: return "NOTIFY"
    if s < t_refuse: return "APPROVE"
    return "REFUSE"

# ---- three worked actions ----
actions = [
    ("send templated email", 0.5, 0.0, 0),
    ("issue $40 refund", 3.0, 40.0, 1),
    ("delete account", 24.0, 800.0, 3),
]
print("action hrs $blast reg r b g score tier")
for name, h, d, reg in actions:
    r, b, g, s = risk_score(h, d, reg)
    print(f"{name:22s} {h:5.1f} {d:6.0f} {reg:4d} "
          f"{r:.3f} {b:.3f} {g:.3f} {s:.4f} {to_tier(s)}")
```

```python

# ---- threshold-crossing: same refund, rising dollar amount (rev=3h, reg=1) ----
print("\nrefund amount crossing NOTIFY -> APPROVE:")
for d in (40, 300, 600):
    r, b, g, s = risk_score(3.0, d, 1)
    print(f" refund ${d:4d}: b={b:.3f} score={s:.4f} -> {to_tier(s)}")

# ---- governance dial 1: raise the blast weight ----
print("\nweight dial (rev=1h, $500, reg=0):")
for (wr, wb, wg) in [(0.30, 0.40, 0.30), (0.15, 0.70, 0.15)]:
    r, b, g, s = risk_score(1.0, 500.0, 0, wr, wb, wg)
    print(f" w={wr:.2f}/{wb:.2f}/{wg:.2f}: score={s:.4f} -> {to_tier(s)}")

# ---- governance dial 2: lower the APPROVE threshold ----
r, b, g, s300 = risk_score(3.0, 300.0, 1)
print(f"\nthreshold dial ($300 refund, score={s300:.4f}):")
print(f" APPROVE-cut=0.35 -> {to_tier(s300, T_NOTIFY, 0.35)}")
print(f" APPROVE-cut=0.25 -> {to_tier(s300, T_NOTIFY, 0.25)}")

# Expected output:
# action hrs $blast reg r b g score tier
# send templated email 0.5 0 0 0.021 0.000 0.000 0.0062 AUTO
# issue $40 refund 3.0 40 1 0.125 0.040 0.333 0.1535 NOTIFY
# delete account 24.0 800 3 1.000 0.800 1.000 0.9200 REFUSE
#
# refund amount crossing NOTIFY -> APPROVE:
# refund $ 40: b=0.040 score=0.1535 -> NOTIFY
# refund $ 300: b=0.300 score=0.2575 -> NOTIFY
# refund $ 600: b=0.600 score=0.3775 -> APPROVE
#
# weight dial (rev=1h, $500, reg=0):
# w=0.30/0.40/0.30: score=0.2125 -> NOTIFY
# w=0.15/0.70/0.15: score=0.3562 -> APPROVE
#
# threshold dial ($300 refund, score=0.2575):
# APPROVE-cut=0.35 -> NOTIFY
# APPROVE-cut=0.25 -> APPROVE
```

— end of document —